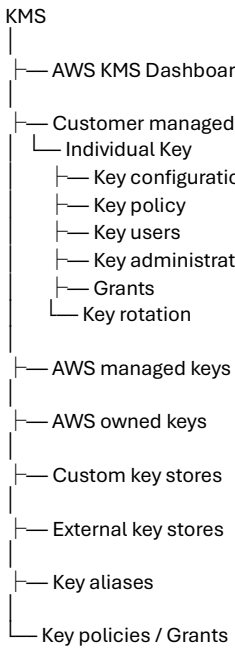
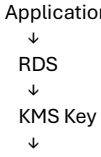
KMS — Console Dashboard Wise Important Interview Questions & Answers



1. What is AWS KMS?
Answer:
AWS KMS (Key Management Service) is used to create and manage encryption keys in AWS.
It helps us protect data stored in services such as S3, EBS, RDS, and other AWS services.
KMS also controls who can use the encryption keys through IAM policies and key policies.
Example:
Suppose our RDS database contains customer information.
We can use a KMS key to encrypt the database and control which users or roles can use that key.
Diagram:



Simple ga gurthupettuko:
KMS = Create + Manage + Control Encryption Keys

---

2. What is a Customer Managed Key?
Answer:
A Customer Managed Key is a KMS key that we create and manage ourselves.
We can control its key policy, rotation, permissions, and lifecycle.
It gives us more control compared with AWS managed keys.
Example:
Suppose our company wants complete control over encryption for production databases.
We can create a customer managed KMS key and allow only specific administrators and applications to use it.
Diagram:



Simple ga gurthupettuko:
Customer Managed Key = More control

---

3. What is an AWS Managed Key?
Answer:
An AWS Managed Key is a KMS key created and managed by AWS for a specific AWS service.
AWS handles the key management, while the customer can use the key through the supported service.
These keys are commonly used when we want AWS-managed encryption without creating our own key.

Example:
Suppose an AWS service supports encryption using an AWS managed key.
We can select the AWS managed key instead of creating and managing a customer managed key ourselves.
Diagram:
AWS Service
↓
AWS Managed Key
↓
Encryption
Simple ga gurthupettuko:
AWS Managed Key = AWS manages the key

---

4. What is an AWS Owned Key?
Answer:
AWS owned keys are encryption keys that AWS owns and manages internally for AWS services.
Customers generally do not manage these keys directly in their AWS account.
They are used by AWS services when encryption is enabled using AWS-owned infrastructure.
Example:
Suppose an AWS service encrypts customer data automatically using AWS-owned keys.
We get encryption protection without having to create or manage a KMS key ourselves.
Diagram:
AWS Service
↓
AWS Owned Key
↓
Encrypted Data
Simple ga gurthupettuko:
AWS Owned Key = AWS owns + manages it

---

5. What is a KMS Key Policy?
Answer:
A key policy is a resource-based policy attached directly to a KMS key.
It defines who can administer the key and who can use the key for encryption or decryption.
KMS key policies are an important part of controlling access to KMS keys.
Example:
Suppose only the security team should manage a KMS key, while an application role can only use it for encryption and decryption.
We can define these permissions using the KMS key policy.
Diagram:
KMS Key
↓
Key Policy
↓

Admin → Manage
App → Use Key
Simple ga gurthupettuko:
Key Policy = Who can manage/use the KMS key

---

6. What is Key Rotation?
Answer:
Key rotation means periodically changing the cryptographic key material used by a KMS key.
It helps follow security best practices without requiring applications to change the KMS key identifier they use.
For supported KMS keys, automatic rotation can be configured from the KMS console.
Example:
Suppose our production application uses a customer managed KMS key.
We can enable automatic rotation so AWS periodically rotates the key material while applications continue using the same KMS key.
Diagram:
KMS Key
↓
Rotation Enabled
↓
New Key Material
↓
Continue Encryption
Simple ga gurthupettuko:
Key Rotation = Periodically rotate key material

---

7. What is a KMS Key Alias?
Answer:
A KMS alias is a friendly name that points to a KMS key.

Instead of remembering a long key ID or ARN, applications and administrators can use an easy-to-understand alias.
Aliases are especially useful when managing multiple encryption keys.
Example:
Suppose we have a production encryption key.
Instead of remembering its long key ID, we can create an alias such as alias/prod-key and use that name to identify the key.
Diagram:
alias/prod-key
↓
KMS Key
↓
Encryption
Simple ga gurthupettuko:
Alias = Friendly name for KMS key

---

8. What is a KMS Grant?
Answer:
A KMS Grant provides specific permissions to a principal for using a KMS key.
It is commonly used by AWS services and applications that need temporary or controlled access to a key.
Grants are different from key policies and IAM policies because they provide another mechanism for delegating KMS permissions.
Example:
Suppose an AWS service needs permission to use a KMS key for a particular operation.
A grant can provide that service the required permission without changing the main key policy.
Diagram:
KMS Key
↓
Grant
↓
AWS Service / Principal
↓
Use Key
Simple ga gurthupettuko:
Grant = Give specific KMS permission

---

9. What are Key Administrators?
Answer:
Key administrators are principals who have permissions to manage a KMS key.
They can perform administrative operations such as changing key settings or managing access, depending on the policy.
They are different from key users, who primarily use the key for cryptographic operations.
Example:
Suppose our security team manages production encryption keys.
We can give the security team's IAM role administrative permissions while giving the application role only key-usage permissions.
Diagram:
KMS Key
↓
Key Administrators
↓
Manage Key

Key Users
↓
Use Key
Simple ga gurthupettuko:
Administrator = Manage Key
User = Use Key

---

10. What are Key Users?
Answer:
Key users are principals that are allowed to use a KMS key for cryptographic operations.
Depending on permissions, they may encrypt, decrypt, generate data keys, or perform other supported operations.
They do not automatically get permission to administer the key.
Example:
Suppose an EC2 application needs to decrypt encrypted application data.
We can allow its IAM role to use the KMS key without giving that role permission to change the key configuration.
Diagram:
EC2
↓
IAM Role
↓
KMS Key
↓
Decrypt Data

Simple ga gurthupettuko:
Key User = Use the key, not necessarily manage it

---

11. What are KMS Custom Key Stores?
Answer:
A custom key store allows KMS keys to use key material stored in an external key management system that integrates with KMS.
It provides additional control over where the cryptographic key material is stored.
Custom key stores are mainly used when an organization has specific security or compliance requirements.
Example:
Suppose a company has strict compliance requirements about controlling where key material is stored.
They can use a KMS custom key store integrated with supported external key management infrastructure.
Diagram:
AWS KMS
↓
Custom Key Store
↓
External Key Material
↓
Encryption
Simple ga gurthupettuko:
Custom Key Store = More control over key material

SQS — Console Dashboard Wise Important Interview Questions & Answers
SQS
│
├── Queues
│ └── Individual Queue
│ ├── Queue configuration
│ ├── Access policy
│ ├── Encryption
│ ├── Dead-letter queue
│ ├── Visibility timeout
│ ├── Message retention
│ ├── Delivery delay
│ └── Long polling
│
├── Standard Queues
├── FIFO Queues
└── Dead-letter Queues
1. What is Amazon SQS?
Answer:
Amazon SQS (Simple Queue Service) is a managed message queuing service.
It allows one application to send messages to a queue while another application processes them later.
This helps separate application components and handle temporary traffic spikes.
Example:
Suppose users place orders on an application.
The application puts each order into an SQS queue, and a worker application reads the messages and processes the orders.
Diagram:
User
↓
Application
↓
SQS Queue
↓
Worker
↓
Process Order
Simple ga gurthupettuko:
SQS = Store messages until a consumer processes them

---

2. What is a Standard Queue?
Answer:
A Standard Queue is designed for high throughput and distributed message processing.
It provides at-least-once message delivery, so duplicate messages can occur.
Applications using Standard Queues should therefore handle duplicate processing safely.
Example:
Suppose an application receives thousands of image-processing requests.
We can use a Standard Queue to distribute those requests among multiple worker applications.
Diagram:
Application

↓
Standard SQS
↓

↓ ↓ ↓
Worker Worker Worker
Simple ga gurthupettuko:
Standard = High throughput + duplicates possible

---

3. What is a FIFO Queue?
Answer:
A FIFO queue is designed when message ordering and exactly-once processing requirements are important.
FIFO means First-In-First-Out, so messages are processed in order within the applicable message group.
It also supports deduplication to help prevent duplicate messages.
Example:
Suppose a banking application receives transactions that must be processed in order.
We can use a FIFO queue so related transactions are processed in the required sequence.
Diagram:
Message 1
↓
Message 2
↓
Message 3
↓
FIFO Queue
↓
Consumer
Simple ga gurthupettuko:
FIFO = Order matters

---

4. What is Visibility Timeout?
Answer:
Visibility timeout is the period during which a message becomes invisible to other consumers after one consumer receives it.
It gives the consumer time to process the message before another consumer can receive it again.
If processing fails and the message is not deleted, it can become visible again after the timeout.
Example:
Suppose a worker receives an order message and needs 60 seconds to process it.
We can configure a suitable visibility timeout so another worker does not immediately process the same message.
Diagram:
SQS Message
↓
Consumer receives
↓
Invisible
↓
Process
↓
Delete Message
Simple ga gurthupettuko:
Visibility Timeout = Temporary hide message after receiving

---

5. What is Message Retention Period?
Answer:
Message retention period defines how long SQS keeps a message in the queue before automatically deleting it.
It allows consumers enough time to process messages even if they are temporarily unavailable.
The retention period can be configured according to the application's requirement.
Example:
Suppose our worker application is temporarily down for a few hours.
Messages can remain in the SQS queue during the configured retention period and be processed when the worker returns.
Diagram:
Producer
↓
SQS Queue
↓
Message Retained
↓
Consumer Available
↓
Process
Simple ga gurthupettuko:
Retention = How long SQS keeps the message

6. What is a Dead-Letter Queue?
Answer:
A Dead-Letter Queue (DLQ) stores messages that could not be successfully processed after a configured number of attempts.
It prevents repeatedly failing messages from continuously blocking normal processing.
DLQs are useful for troubleshooting and handling failed messages separately.
Example:
Suppose a worker tries to process an order message several times but the message always fails.
After the configured retry limit, SQS can move that message to a DLQ for investigation.
Diagram:
SQS Queue
↓
Processing Failed
↓
Retry
↓
Retry Limit
↓
Dead-Letter Queue
Simple ga gurthupettuko:
DLQ = Failed messages ki separate queue

---

7. What is Long Polling in SQS?
Answer:
Long polling allows SQS to wait for messages to become available instead of immediately returning an empty response.
This can reduce unnecessary empty receive requests and improve efficiency.
It is commonly preferred when consumers are continuously waiting for new messages.
Example:
Suppose our worker continuously checks SQS for new orders.
With long polling, it can wait for messages instead of repeatedly making requests when the queue is empty.
Diagram:
Worker
↓
SQS
↓
Wait for Message
↓
Message Arrives
↓
Return Message
Simple ga gurthupettuko:
Long Polling = Wait for message instead of frequent empty checks

---

8. What is Delay Queue?
Answer:
A delay queue temporarily hides new messages from consumers for a configured period.
This is useful when we do not want a message to be processed immediately after it is sent.
The message becomes available after the delay period.
Example:
Suppose an application wants to process a task 5 minutes after it is created.
A delay queue can keep the message hidden for the configured delay period before making it available.
Diagram:
Producer
↓
SQS
↓
Delay Period
↓
Message Available
↓
Consumer
Simple ga gurthupettuko:
Delay Queue = Process message after some time

---

9. How does SQS encryption work?
Answer:
SQS supports server-side encryption to protect messages stored in the queue.
We can use AWS-managed encryption or a customer managed KMS key depending on the queue and security requirement.
Encryption protects message data while it is stored by SQS.

Example:
Suppose our queue contains sensitive customer information.
We can enable server-side encryption so the messages stored in the queue are encrypted.
Diagram:
Producer
↓
SQS
↓
KMS Encryption
↓
Encrypted Message
Simple ga gurthupettuko:
SQS Encryption = Protect messages at rest

VPC Deep Networking — Important Console Areas
VPC
│
├— Your VPCs
├— Subnets
├— Route Tables
├— Internet Gateways
├— Egress Only Internet Gateways
├— NAT Gateways
├— Security Groups
├— Network ACLs
├— VPC Endpoints
├— Peering Connections
├— Network Firewall
├— DHCP Option Sets
└— Flow Logs
1. What is a Route Table?
Answer:
A route table contains rules that decide where network traffic should be sent.
Each route has a destination and a target such as an Internet Gateway, NAT Gateway, or another network component.
Subnets are associated with route tables to control their traffic path.
Example:
Suppose a public subnet needs internet access.
We can associate it with a route table containing a default route to an Internet Gateway.
Diagram:
Subnet
↓
Route Table
↓
0.0.0.0/0
↓
Internet Gateway
Simple ga gurthupettuko:
Route Table = Where traffic should go

---

2. What is a NAT Gateway?
Answer:
A NAT Gateway allows resources in a private subnet to access the internet without allowing the internet to directly initiate connections to those resources.
It is normally placed in a public subnet and uses an Elastic IP for internet connectivity.
Private subnet traffic is routed through the NAT Gateway.
Example:
Suppose an EC2 instance in a private subnet needs to download software updates.
We can route its internet-bound traffic through a NAT Gateway without giving the EC2 instance a public IP.
Diagram:
Private EC2
↓
Route Table
↓
NAT Gateway
↓
Internet Gateway
↓
Internet
Simple ga gurthupettuko:
NAT = Private subnet → Internet

---

3. What is a VPC Endpoint?
Answer:
A VPC Endpoint allows resources inside a VPC to access supported AWS services without using the public internet.
It can improve security by keeping traffic on the AWS network.
Common endpoint types include Gateway endpoints and Interface endpoints.
Example:
Suppose an EC2 instance in a private subnet needs to access S3.
We can use an S3 Gateway Endpoint so the traffic does not need to go through a NAT Gateway.
Diagram:
Private EC2
↓
VPC Endpoint
↓
AWS Service
↓
S3
Simple ga gurthupettuko:
VPC Endpoint = Private connection to AWS services

---

4. What is a Network ACL?
Answer:
A Network ACL (NACL) is a network-level access control mechanism applied to a subnet.
It controls inbound and outbound traffic using allow and deny rules.
Unlike Security Groups, NACLs are stateless, so inbound and outbound rules are evaluated separately.
Example:
Suppose we want to block traffic from a specific IP range at the subnet level.
We can add a deny rule in the Network ACL to block that traffic.
Diagram:
Traffic
↓
Network ACL
↓
Subnet
↓
Resources
Simple ga gurthupettuko:
NACL = Subnet-level traffic control

---

5. What is a Security Group?
Answer:
A Security Group acts as a virtual firewall for resources such as EC2 instances.
It controls inbound and outbound traffic using security group rules.
Security Groups are stateful, so return traffic is automatically allowed when the corresponding request is allowed.
Example:
Suppose an EC2 web server should accept HTTP traffic from users but SSH only from an administrator IP.
We can configure the Security Group with separate rules for HTTP and SSH access.
Diagram:
Internet
↓
Security Group
↓
EC2
Simple ga gurthupettuko:
Security Group = Resource-level stateful firewall

Aurora — Console Dashboard Wise Important Interview Questions & Answers
Aurora
│
├── Databases
│ └── Aurora Cluster
│ ├── Connectivity & security
│ ├── Monitoring
│ ├── Logs & events
│ ├── Configuration
│ ├── Maintenance
│ └── Backup
│
├── Snapshots
├── Automated Backups

├— Subnet Groups
├— Parameter Groups
├— Option Groups
├— Performance Insights
└— Blue/Green Deployments

1. What is Amazon Aurora?

Answer:

Amazon Aurora is a managed relational database engine compatible with MySQL and PostgreSQL.
It is designed to provide high performance and high availability with AWS-managed database infrastructure.
Aurora uses a cluster architecture with shared distributed storage.

Example:

Suppose our application needs a highly available relational database.
Instead of managing MySQL servers ourselves, we can create an Aurora cluster and let AWS manage much of the database infrastructure.

Diagram:

Application
↓
Aurora Cluster
/ \
Writer Reader
\ /
Shared Storage

Simple ga gurthupettuko:

Aurora = AWS managed high-performance relational database

---

2. What is an Aurora Cluster?

Answer:

An Aurora cluster is a group of database instances connected to Aurora's shared storage layer.
It normally contains a writer instance and can contain one or more reader instances.
This architecture supports high availability and read scaling.

Example:

Suppose our application has heavy database traffic.
We can have one Aurora writer for updates and additional Aurora readers for read-heavy application requests.

Diagram:

Aurora Cluster
↓

↓ ↓
Writer Reader
↓
Read Traffic

Simple ga gurthupettuko:

Aurora Cluster = Writer + Readers + Shared Storage

---

3. What is an Aurora Writer Instance?

Answer:

The Aurora writer instance is the primary database instance that handles write operations.
Applications send INSERT, UPDATE, and DELETE operations to the writer.
If the writer fails, Aurora can fail over to a suitable reader instance.

Example:

Suppose users place orders in our application.
The order information is written to the Aurora writer instance, while reader instances can handle read requests.

Diagram:

Application
↓
Aurora Writer
↓
Shared Aurora Storage

Simple ga gurthupettuko:

Writer = Handles database writes

---

4. What is an Aurora Reader Instance?

Answer:

An Aurora reader instance is used mainly for read operations.
It can help distribute read traffic and improve application scalability.
Reader instances can also provide failover targets for the Aurora cluster.

Example:

Suppose thousands of users are viewing products at the same time.
We can send read requests to Aurora reader instances while the writer handles database changes.

Diagram:

Users

↓
Read Traffic
↓
Aurora Reader
↓
Shared Storage
Simple ga gurthupettuko:
Reader = Read scaling + Failover support

---

5. What is the Aurora Cluster Endpoint?
Answer:
The cluster endpoint is used to connect applications to the primary/writer side of an Aurora cluster.
It automatically points to the current writer instance.
This makes failover easier because applications do not need to manually change the database hostname.
Example:
Suppose the current Aurora writer fails and another instance becomes the writer.
The cluster endpoint automatically points to the new writer, so the application can continue using the same endpoint.
Diagram:
Application
↓
Cluster Endpoint
↓
Current Writer
Simple ga gurthupettuko:
Cluster Endpoint = Connect to current Writer

---

6. What is the Aurora Reader Endpoint?
Answer:
The reader endpoint is used to distribute read connections across Aurora reader instances.
It is useful when an application has many read operations.
Aurora can route connections to available reader instances.
Example:
Suppose our application mostly performs SELECT queries.
We can configure the application to use the reader endpoint so read traffic can be distributed across Aurora readers.
Diagram:
Application
↓
Reader Endpoint
↙ ↓ ↘
Reader Reader Reader
Simple ga gurthupettuko:
Reader Endpoint = Connect to Aurora readers

---

7. What is Aurora Backup?
Answer:
Aurora automatically backs up cluster data to Amazon S3 as part of its managed backup system.
These backups support recovery to a point in time within the configured retention period.
Aurora also supports manual snapshots for longer-term or specific recovery needs.
Example:
Suppose a developer accidentally changes important database data.
We can use Aurora's backup capabilities to restore the database to an appropriate point in time.
Diagram:
Aurora Cluster
↓
Automatic Backup
↓
Amazon S3
↓
Point-in-Time Recovery
Simple ga gurthupettuko:
Aurora Backup = Managed backup + Recovery

---

8. What is Aurora Failover?
Answer:
Aurora failover moves the writer role to another suitable database instance when the current writer becomes unavailable.
This helps maintain database availability during failures.
Applications using the appropriate cluster endpoint can reconnect to the new writer.
Example:
Suppose the Aurora writer instance suddenly fails.
Aurora can promote an available reader and the cluster endpoint can direct connections to the new writer.